Модуль 2. Задание №2. Вёрстка страницы HTML+CSS+JAVASCRIPT

Разберем пример вёрстки страницы с помощью HTML, CSS и JAVASCRIPT, где HTML является основным «скелетом» элементов (тегов), CSS задает стили каждому элементу, а JAVASCRIPT будет «рендерить» содержимое (контент) внутрь HTML-блоков.

Пример проекта

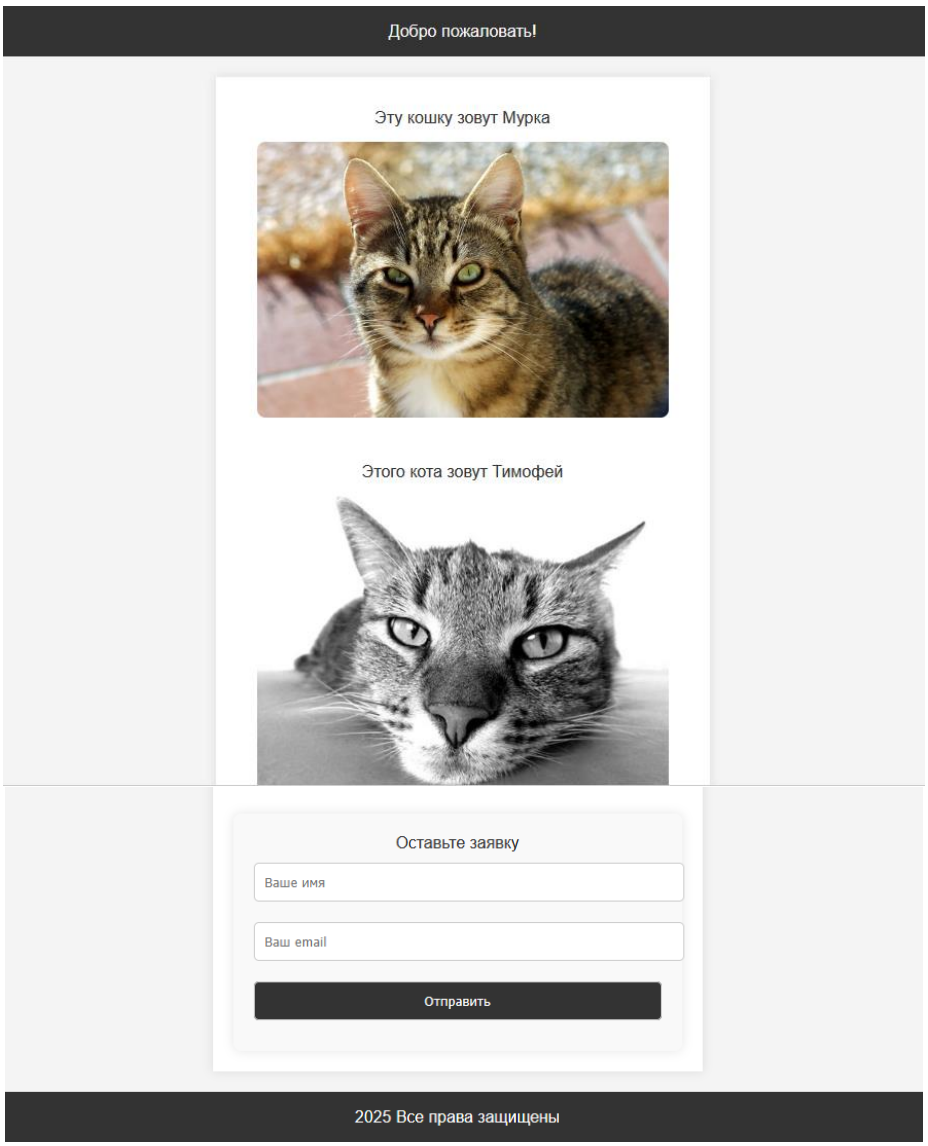

Рисунок 1 – Эталонный пример макета

1. Выполните чистую верстку с заданиями.

1.1) Переписываем HTML-структуру проекта (ссылки на картинки котиков берем с интернета). Названия (классы) элементов рекомендуется сделать такими же, как и на картинке.

index.html > ...

```html
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JS</title>
    <link rel="stylesheet" href="./style.css">
</head>
<body>
    <header>
            <h1>Добро пожаловать!</h1>
    </header>
    <main class="container">
        <div class="text-block">
            <p>Эту кошку зовут Мурка</p>
            <img class="image" src="https://upload.wikimedia.org/wikipedia/commons/thumb/7/74/A-Cat.jpg/2560px-A-Cat.jpg" alt="Приме
        </div>
        <div class="text-block">
            <p>Этого кота зовут Тимофей</p>
            <img class="image" src="https://images.all-free-download.com/images/thumbjpg/cat_hangover_relax_213869.jpg" alt="Пример
        </div>
        <div class="form-container">
            <h3>Передайте привет</h3>
            <form>
                <input type="text" placeholder="От кого">
                <input type="text" placeholder="Кому">
                <button type="submit">Отправить</button>
            </form>
        </div>
    </main>
        <footer>
            2025 Все права защищены
        </footer>

    <script src="./script.js"></script>
</body>
</html>
```

2. Работа со стилями CSS.

2.1) Во-первых, вставьте код для сброса параметров стилей (reset css). (https://meyerweb.com/eric/tools/css/reset/).

```
# style.css ×
# style.css > html
html, body, div, span, applet, object, iframe,
h1, h2, h3, h4, h5, h6, p, blockquote, pre,
a, abbr, acronym, address, big, cite, code,
del, dfn, em, img, ins, kbd, q, s, samp,
small, strike, strong, sub, sup, tt, var,
b, u, i, center,
dl, dt, dd, ol, ul, li,
fieldset, form, label, legend,
table, caption, tbody, tfoot, thead, tr, th, td,
article, aside, canvas, details, embed,
figure, figcaption, footer, header, hgroup,
menu, nav, output, ruby, section, summary,
time, mark, audio, video {
    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    font: inherit;
    vertical-align: baseline;
}
article, aside, details, figcaption, figure,
footer, header, hgroup, menu, nav, section {
    display: block;
}
```

2.2) Далее необходимо правильно подставить названия элементов (CSS-селекторы) для корректной работы свойств.

Пример: Переписываем стили для элемента body:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    align-items: center;
}
```

2.3) Далее самостоятельно подставьте правильные названия 8 элементов для стилей:

```
[redacted]
background: #333;
color: white;
text-align: center;
padding: 1rem;
width: 100%;
```

```
[redacted]
text-align: center;
padding: 10px;
}
```

```
[redacted]
margin-top: 15px;
max-width: 400px;
height: auto;
border-radius: 8px;
}
```

```
[redacted]
max-width: 800px;
margin: 20px auto;
padding: 20px;
background: white;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
align-items: center;
gap: 20px;
```

```
[redacted]
text-align: center;
padding: 1rem;
background: #333;
color: white;
width: 100%;
margin-top: auto;
}
```

```
[redacted] {
background-color: #333;
color: white;
cursor: pointer;
}
```

```
width: 100%;
padding: 10px;
margin: 10px 0;
border: 1px solid #ccc;
border-radius: 5px;
```

```
[redacted] {
width: 100%;
max-width: 400px;
padding: 20px;
background: #f9f9f9;
border-radius: 8px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
text-align: center;
}
```

2.4) Проверьте результат работы в браузере (откройте файл index.html через браузер). У вас должен получиться результат, как на рисунке 1.

3. Вам необходимо выполнить то же самое, но с применением JavaScript. Выполняем пошагово с небольшими заданиями.

HTML -> JS.

После того, как выполнили верстку страницы с котиками и формой, проверьте, чтобы получился такой же результат, как и на первой иллюстрации. Сделайте копию текущего проекта, чтобы потом сравнивать результаты.

Переходим в копию проекта и подключаем файл скриптов к нашей странице (<script src="./script.js">). Основная структура страницы делится на 3 основных блока (header, main, footer). Стили проекта остаются теми же. Ваша задача написать 3 основные функции, которые будут рендерить блоки и контент в них, а также задавать атрибуты классов для применения готовых стилей.

1) createHeader();
2) createMain();
3) reateFooter();

3.1) Очищаем HTML-разметку, оставляем только основной блок <body>. Стили (файл style.css) остается неизменным.

```html
index.html > html > body
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JS</title>
    <link rel="stylesheet" href="./style.css">
</head>
<body>

    <script src="./script.js"></script>
</body>
</html>
```

3.2) Разработка HEADER. Пример: Код блока <header>. Описываем функцию createHeader(), которая создает элемент <header>, внутри которой создается элемент (заголовок) <h1> фразой «Добро пожаловать!». Обратите внимания, что атрибут CLASS задавать данному блоку не нужно, т.к. стили прописаны глобально для всех элементов header. Не забывайте выполнить вызов функции createHeader().

```js
JS script.js > createMain
function createHeader() {
    const header = document.createElement("header");
    header.innerHTML = "<h1>Добро пожаловать!</h1>";
    document.body.appendChild(header);
}
```

```js
createHeader();
createMain();
createFooter();
```

Результат должен быть таким же, как и на исходном варианте.

Добро пожаловать!

3.3) Разработка FOOTER. САМОСТОЯТЕЛЬНО реализуйте функцию createFooter(). Она строится аналогичным образом, как и createHeader(). Не забудьте написать правильное название элементов, чтобы к ним применились стили.

3.4) Разработка MAIN. MAIN является самым сложным блоком. Он содержит в себе: Блоки с текстом, картинками и формой.

Предлагается следующий принцип разработки блока: главная функция createMain() вызывает в себе две функции: createImgBlock() (блок с именем кота и картинкой) и createForm() (блок с формой).

3.5) На рисунке ниже представлен код реализации функции. Подумайте, какой функции он принадлежит и что нужно добавить. Перепишите код правильно.

```
function [redacted] {
    const main = document.createElement([redacted]);
    main.appendChild(createImgBlock("Это кот 1", "https://upload.wikime
    /* здесь создаем второй блок с котиком*/
    [redacted]appendChild(createForm());
    document.body.appendChild[redacted]
}
```

- appendChild() добавляет элементы внутрь других элементов.
- document.body.appendChild(main); добавляет весь <main> в body.

Дополнительные функции необходимо разработать ниже.

3.6) Необходимо описать оставшиеся 2 функции: createImgBlock() и createForm()

3.7) Разработка функции createImgBlock(). Подумайте, какой элемент нужно создать с помощью функции createElement(“ ”). Какое имя класса будет для стилей данного элемента. И какой результат должна возвращать данная функция. Перепишите код правильно.

```
function createImgBlock(text, imgSrc) {
    const block = document.createElement([redacted]
    block.className = [redacted];
    block.innerHTML = `<p>${text}</p>
    <img class='image' src='${imgSrc}' alt='Пример'>`;
    return [redacted];
}
```

Как работает функция createImgBlock(). Не забудьте добавить в функцию 2 параметра: **text** (текст для каждой картинки), **imgSrc** (путь к каждому файлу картинки).

1. Создает , который будет контейнером для изображения и текста.
2. Добавляет класс text-block для стилизации.
3. Заполняет содержимое через innerHTML:

- ${text} — абзац с переданным текстом.
- <img> — изображение с переданной ссылкой imgSrc.

4. Возвращает созданный элемент (return block), чтобы его можно было добавить в <main>.

В JavaScript конструкция `${text}` внутри строки с обратными кавычками (`...`) называется **шаблонными строками (template literals**). Это удобный способ вставлять переменные в текст, не используя конкатенацию (+).

В нашем случае:

- ${text} – вставляет переданный текст в абзац .
- ${imgSrc} – подставляет ссылку на изображение в src.

Пример:

Если мы вызовем функцию так: createImgBlock("Котик спит", "https://example.com/cat.jpg");

То получится следующий результат:

```
<p>Котик спит</p>
<img class='image'
  src='https://example.com/cat.jpg'
  alt='Пример'>
```

3.8) Разработка функции createForm(). По тому же принципу разрабатывается данная функция, но функции параметры задавать не нужно. Подумайте, какой элемент должен создаваться, в который будут рендериться форма (fomr) с двумя input полями. Задайте правильное имя класса и вызовите необходимую функцию для заполнения контента. Внутри данной функции создайте заголовок (h3), форму, input-поля, а также кнопку (button) «отправить». Не забудьте вернуть правильное значение функции.

```
function createForm() {
    const formContainer = document.createElement([redacted]);
    formContainer.className = [redacted];
    formContainer.[redacted] = `
        <h3>Оставьте заявку</h3>
        [redacted]
            <input type="text" placeholder="Ваше имя" required>
            <input type="email" placeholder="Ваш email" required>
            [redacted]
        [redacted]
    `;
    return [redacted]
}
```

4. CSS -> JS.

Для усложнения задания выполните те же самые процедуры со стилями. Закомментируйте (/*) все стили в файле style.css и пропишите все свойства стилей для каждого элемента ваших JS-функций.

```
/*
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    color: #333;
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    align-items: center;
}
header {
    background: #333;
    color: white;
```

Метод Object.assign() позволяет копировать значения всех перечисляемых собственных свойств из одного или более исходных объектов в целевой объект. При использовании данного варианта, перечисление свойств стилей может происходит построчно, что упрощает читаемость кода.

4.1) Добавим и определим функцию applyStyles()

```
function applyStyles(element, styles) {
    Object.assign(element.style, styles);
}
```

4.2) Пример применения стилей на JS для блока HEADER() представлен ниже (свойства стилей можно скопировать из файла style.css):

```
function createHeader() {
    const header = document.createElement("header");
    header.innerHTML = "<h1>Добро пожаловать!</h1>";
    applyStyles(header, {
        background: "#333",
        color: "white",
        textAlign: "center",
        padding: "1rem",
        width: "100%"
    });
    document.body.appendChild(header);
}
```

4.3) Аналогичным образом САМОСТОЯТЕЛЬНО пропишите код функции createFooter() со стилями (свойства стилей скопируйте из файла style.css).

4.4) Разработка функции createMain() со стилями:

```
function createMain() {
    const main = document.createElement("main");
    applyStyles([redacted]{
        maxWidth: "800px",
        margin: "20px auto",
        padding: "20px",
        background: "white",
        boxShadow: "0 0 10px rgba(0, 0, 0, 0.1)",
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        gap: "20px"
    });
    main.appendChild(createImgBlock("Этого кота зовут Тимофей", "htt
    main.appendChild(createImgBlock("Эту кошку зовут Мурка", "https:
    main.appendChild(createForm());
    document.body.appendChild(main);
}
```

4.5) Реализация функции createImgBlock() со стилями:

```
function createImgBlock(text, imgSrc) {
    const block = document.createElement("div");
    applyStyles(block, {
        textAlign: "center",
        padding: "10px"
    });
    block.innerHTML = `<p>${text}</p>`;
    const img = document.createElement("img");
    img.src = imgSrc;
    img.alt = "Пример";
    applyStyles(img, {
        maxWidth: "100%",
        height: "auto",
        borderRadius: "8px"
    });
    block.appendChild(img);
    return block;
}
```

4.6) Реализация функции createForm() со стилями:

```
function createForm() {
    const formContainer = document.createElement("div");
    applyStyles(formContainer, {
        width: "100%",
        maxWidth: "400px",
        padding: "20px",
        background: "#f9f9f9",
        borderRadius: "8px",
        boxShadow: "0 0 10px rgba(0, 0, 0, 0.1)",
        textAlign: "center"
    });
```

```
    formContainer.innerHTML = `
        <h3>Оставьте заявку</h3>
        <form>
            <input type="text" placeholder="Ваше имя" required>
            <input type="email" placeholder="Ваш email" required>
            <button type="submit">Отправить</button>
        </form>
    `;
```

```
    formContainer.querySelectorAll("input, button").forEach(el => {
        applyStyles(el, {
            width: "100%",
            padding: "10px",
            margin: "10px 0",
            border: "1px solid #ccc",
            borderRadius: "5px"
        });
    });
    applyStyles(formContainer.querySelector("button"), {
        backgroundColor: "#333",
        color: "white",
        cursor: "pointer"
    });
    return formContainer;
}
```

Работа завершена! Проверьте результат в браузере. Файл style.css можно удалить ☺